MITRC
MODERN INSTITUTE OF TECHNOLOGY AND RESEARCH CENTRE

Name of Faculty: Chakravarty Sharma

Subject & Subject Code: Introduction to Python Programming & 4CS4-05

Semester: IV **Session:** 2022-23

Unit No.: 2 **Date of Submission:** 14/03/2023

Lecture No.	Topics Covered	Page No.	Total Page
9	Decision making : selective statments- if-else, nested if, if- elif ladder statements.	2- 9	8
10	Decision making related programs	10 - 14	5
11	Iterative statements - while, for, Nested loops, else in loops, break, continue and pass statements,	15-19	5
12	Looping: Loop Contro statement – math and random number functions	20-25	6
13	Looping related programs	26-30	5
14	User- defind functions - functions argument & its types.	31-34	4
15	Regular expresion: Matching the patterns, search & replace	35-39	5
16	Practice of programs	40-44	5

References:

1. Programming Python ny Mark Lutz, O`Reilly
2. Intrduction to Programming using Python by Y.Daniel Liang, Pearson, 2012

14/3/2023

Faculty **HOD** **Dean Academics**

Lecture No. 09

Decision Making in Python using if, if-else, if-elif and nested statements

Decision-Making Statements in Python
There comes a point in your life where you need to decide what steps should be taken and based on that you decide your next decisions.

In programming, we often have this similar situation where we need to decide which block of code should be executed based on a condition..

Python has the following decision-making statements:

- if statements
- if-else statements
- if-elif ladder
- Nested statements

Python if statement
if statement is the most simple form of decision-making statement. It takes an expression and checks if the expression evaluates to True then the block of code in if statement will be executed.
If the expression evaluates to False, then the block of code is skipped.

Syntax:

```
if ( Logical Expression ):
        Statement 1
        Statement 2
        ……
        Statement n
```

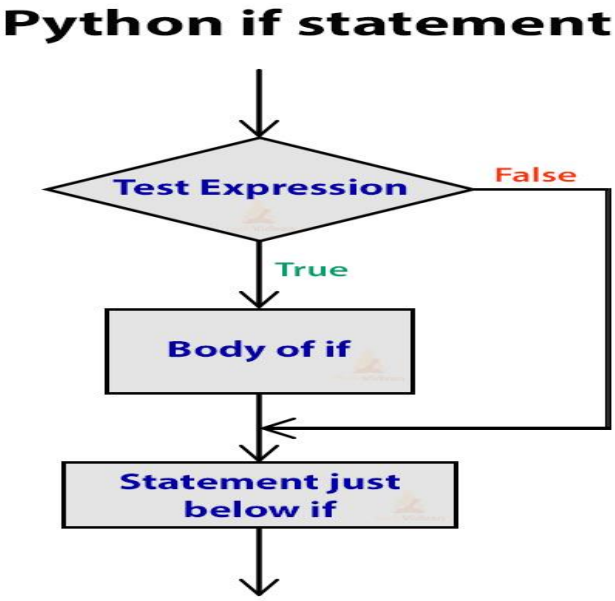
Example

```
a=int(input("Enter any number "))
if a % 2==0:
        print("Even Number")
else:
        print("Odd Number")
```

Note : The if-else statement can be nested according to programming need.

Syntax:

```
if  condition-1:
        statement-1
else:
        if codition-2:
                statement-2
        else:
                ….
```



Example:

```
a = 20 ; b = 20
if ( a == b ):
  print( "a and b are equal")
print("If block ended")
```

Output:

```
a and b are equal
If block ended
```

Example:

```
num = 5
if ( num >= 10):
  print("num is greater than 10")
print("if block ended")
```

Output:

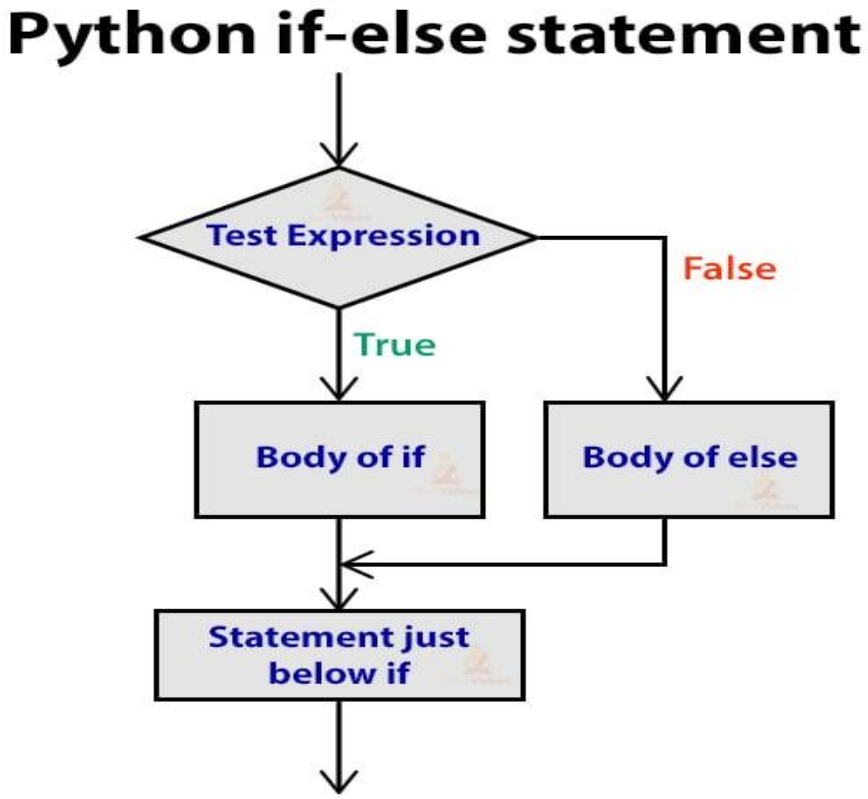
```
If block ended
```

Python if-else statement

The if-else statement checks the expression and executes the if block when the expression is True otherwise it will execute the else block of code. The else block should be right after if block and it is executed when the expression is False.

Syntax:

```
If ( expression ):
  Statement
else:
  Statement
```



Example
number1 = 20 ; number2 = 30
if(number1 >= number2):
  print("number 1 is greater than number 2")
else:
  print("number 2 is greater than number 1")
Output:
number 2 is greater than number 1

Note: Only one else statement is followed by an if statement. If you use two else statements after an if statement, then you get the following error.

Example:
if (5>10):
  print(5)
else:
  print(10)
else:
  print("End")
Output:
SyntaxError: invalid syntax

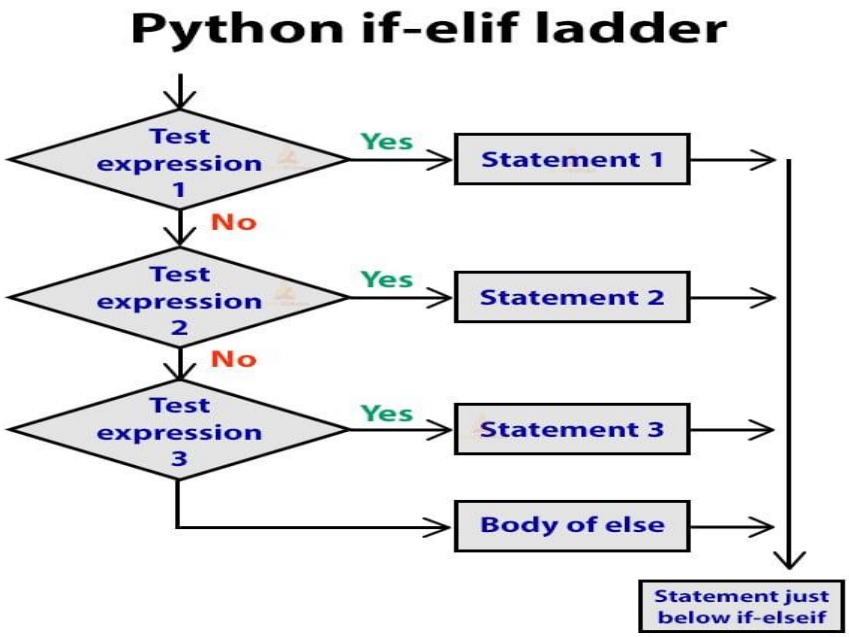
Python if-elif ladder

In Python, we have an elif keyword to chain multiple conditions one after another with elif ladder, we can make complex decision-making statements.

The elif statement helps you to check multiple expressions and it executes the code as soon as one of the conditions evaluates to True.

Syntax:

```
if( expression1 ):
  statement
elif (expression2 ) :
  statement
elif(expression3 ):
  statement
.
.
else:
  statement
```



Example:

```
a=int(input("Enter number "))
if a==1:
   print("sc")
elif a==2:
   print("OBC")
elif a==3:
   print("Gen")
else:
   print("Other")
```

Find result

```
roll_no=int(input("Enter Roll Number : "))
student_name=input("Enter Student Name : ")
regular=bool(input("Enter True or False "))
obt_mark=float(input("Enter Obtained Marks / 500 : "))
per=obt_mark*100/500
if per>=50:
   Result="Pass"
else:
   Result="Fail"
if per>=70:
   g="A"
else:
   if per>=60:
      g="B"
   else:
      if per>=50:
         g="C"
      else:
         g="Nil"

roll="Roll Number : "
sn="Student Name  : "
reg="Student :"
marks="Obtained Marks :"
p="Calculated Percentage : "
h="STUDENT`S RESULT"
RES="Result : "
gr="Grade  :"
if regular:
   r='Regular'
else:
   r="Non-Regular"
print(h)
print(roll,roll_no)
print(sn,student_name)
print(marks,obt_mark)
print(p,per)
print(gr,g)
print(RES,Result)
print(reg,r)
```

```
Enter Roll Number : 1111
Enter Student Name : Dinesh Kumar Singh
Enter True or False true
```

Enter Obtained Marks / 500 : 345

Output

STUDENT`S RESULT
Roll Number : 1111
Student Name : Dinesh Kumar Singh
Obtained Marks : 345.0
Calculated Percentage : 69.0
Grade : B
Result : Pass
Student : Regular

Example:

```
print("Select your ride:")
print("1. Bike")
print("2. Car")
print("3. SUV")

choice = int( input() )

if( choice == 1 ):
  print( "You have selected Bike" )
elif( choice == 2 ):
  print( "You have selected Car" )
elif( choice == 3 ):
  print( "You have selected SUV" )
else:
  print("Wrong choice!")
```

Output 1:

Select your ride:
1. Bike
2. Car
3. SUV
3
You have selected SUV

Output 2:

Select your ride:
1. Bike
2. Car
3. SUV
10
Wrong choice!

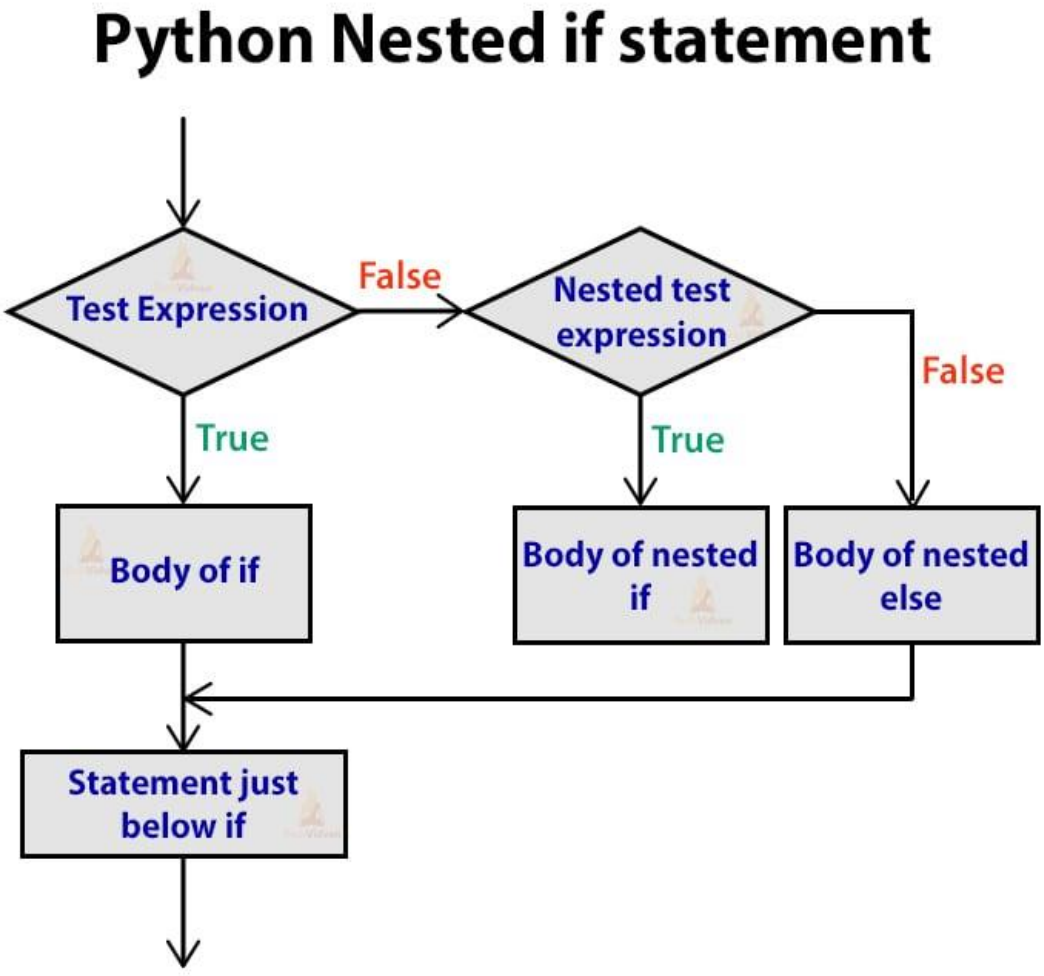
Note: Core Python doesn't support switch-case statements that are available in other programming languages but we can use the elif ladder instead of switch cases.

Python Nested if statement

In very simple words, Nested if statements is an if statement inside another if statement. Python allows us to stack any number of if statements inside the block of another if statements. They are useful when we need to make a series of decisions.

Syntax:

```
if (expression):
  if(expression):
    Statement of nested if
  else:
    Statement of nested if else
  Statement of outer if
Statement outside if block
```



Example :

```
num1 = int( input())
num2 = int( input())
```

```
if( num1>= num2):
   if(num1 == num2):
      print(f'{num1} and {num2} are equal')
   else:
      print(f'{num1} is greater than {num2}')
else:
   print(f'{num1} is smaller than {num2}')
```

Output 1:

```
10
20
10 is smaller than 20
```

Output 2:

```
5
5
5 and 5 are equal
```

Lecture No. 10

Python Loops

Python has two primitive loop commands:

- while loops
- for loops

The while Loop

With the while loop we can execute a set of statements as long as a condition is true.

Example

Print i as long as i is less than 6:

```
i = 1
while i < 6:
  print(i)
  i += 1
```

Note: remember to increment i, or else the loop will continue forever.

The while loop requires relevant variables to be ready, in this example we need to define an indexing variable, i, which we set to 1.

The break Statement

With the break statement we can stop the loop even if the while condition is true:

Example

Exit the loop when i is 3:

```
i = 1
while i < 6:
  print(i)
  if i == 3:
    break
  i += 1
```

The continue Statement

With the continue statement we can stop the current iteration, and continue with the next:

Example

Continue to the next iteration if i is 3:

```
i = 0
while i < 6:
  i += 1
  if i == 3:
    continue
  print(i)
```

The else Statement
With the else statement we can run a block of code once when the condition no longer is true:
Example
Print a message once the condition is false:

```
i = 1
while i < 6:
  print(i)
  i += 1
else:
  print("i is no longer less than 6")
```

It is used when a statement is required syntactically but you do not want any command or code to execute.

The pass statement is a *null* operation; nothing happens when it executes. The **pass** is also useful in places where your code will eventually go, but has not been written yet (e.g., in stubs for example) −
Syntax

```
pass
```

Example

```
for letter in 'Python':
   if letter == 'h':
      pass
      print 'This is pass block'
   print 'Current Letter :', letter

print "Good bye!"
```

When the above code is executed, it produces following result −

```
Current Letter : P
Current Letter : y
Current Letter : t
This is pass block
Current Letter : h
Current Letter : o
Current Letter : n
Good bye!
```

Calculate the sum of digits of any given number with using user defined function

```
def sum(a):
    s=0
    while a!=0:
        s=s+a%10
        a=a//10
    print("Sum of digits ",s)
```

```
num=int(input("Enter any number "))
sum(num)
```

Output

```
Enter any number 567
Sum of digits  18
```

Python for Loop

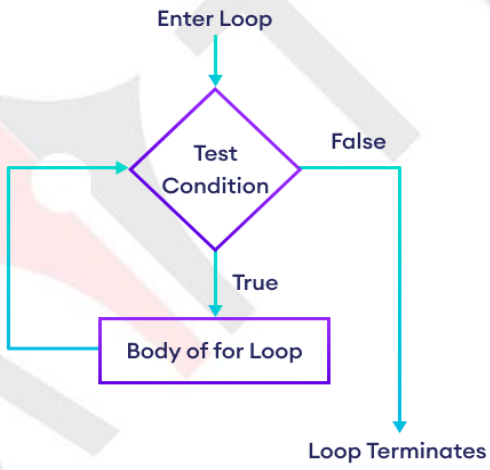
In Python, the for loop is used to run a block of code for a certain number of times. It is used to iterate over any sequences such as list, tuple, string, etc.

The syntax of the for loop is:

```
for val in sequence:
    # statement(s)
```

Here, val accesses each item of sequence on each iteration. Loop continues until we reach the last item in the sequence.

Flowchart of Python for Loop



Working of Python for loop

Example: Loop Over Python List

```
languages = ['Swift', 'Python', 'Go', 'JavaScript']

# access items of a list using for loop
for language in languages:
    print(language)
```

Output
Swift
Python
Go
JavaScript

In the above example, we have created a list called languages.
Initially, the value of language is set to the first element of the array,i.e. Swift, so the print statement inside the loop is executed.
language is updated with the next element of the array and the print statement is executed again.
This way the loop runs until the last element of an array is accessed.

Python for Loop with Python range()

A range is a series of values between two numeric intervals.
We use Python's built-in function range() to define a range of values. For example,
values = range(4)
Here, 4 inside range() defines a range containing values 0, 1, 2, 3.
In Python, we can use for loop to iterate over a range. For example,

```
# use of range() to define a range of values
values = range(4)

# iterate from i = 0 to i = 3
for i in values:
    print(i)
```

Run Code

Output

```
0
1
2
3
```

In the above example, we have used the `for` loop to iterate over a range from **0** to **3**.

The value of `i` is set to **0** and it is updated to the next number of the range on each iteration. This process continues until **3** is reached.

Iteration

ExampleGet your own Python Server

Print each fruit in a fruit list:

```
fruits = ["apple", "banana", "cherry"]
for x in fruits:
  print(x)
```

Lecture No. 11

Python math Module
Python has a built-in module that you can use for mathematical tasks.
The math module has a set of methods and constants.

Method	Description
math.acos()	Returns the arc cosine of a number
math.acosh()	Returns the inverse hyperbolic cosine of a number
math.asin()	Returns the arc sine of a number
math.asinh()	Returns the inverse hyperbolic sine of a number
math.atan()	Returns the arc tangent of a number in radians
math.atan2()	Returns the arc tangent of y/x in radians
math.atanh()	Returns the inverse hyperbolic tangent of a number
math.ceil()	Rounds a number up to the nearest integer
math.comb()	Returns the number of ways to choose k items from n items without repetition and order
math.copysign()	Returns a float consisting of the value of the first parameter and the sign of the second parameter
math.cos()	Returns the cosine of a number
math.cosh()	Returns the hyperbolic cosine of a number
math.degrees()	Converts an angle from radians to degrees
math.dist()	Returns the Euclidean distance between two points (p and q), where p and q are the coordinates of that point
math.erf()	Returns the error function of a number
math.erfc()	Returns the complementary error function of a number
math.exp()	Returns E raised to the power of x

math.expm1()	Returns E^x - 1
math.fabs()	Returns the absolute value of a number
math.factorial()	Returns the factorial of a number
math.floor()	Rounds a number down to the nearest integer
math.fmod()	Returns the remainder of x/y
math.frexp()	Returns the mantissa and the exponent, of a specified number
math.fsum()	Returns the sum of all items in any iterable (tuples, arrays, lists, etc.)
math.gamma()	Returns the gamma function at x
math.gcd()	Returns the greatest common divisor of two integers
math.hypot()	Returns the Euclidean norm
math.isclose()	Checks whether two values are close to each other, or not
math.isfinite()	Checks whether a number is finite or not
math.isinf()	Checks whether a number is infinite or not
math.isnan()	Checks whether a value is NaN (not a number) or not
math.isqrt()	Rounds a square root number downwards to the nearest integer
math.ldexp()	Returns the inverse of math.frexp() which is x * (2**i) of the given numbers x and i
math.lgamma()	Returns the log gamma value of x
math.log()	Returns the natural logarithm of a number, or the logarithm of number to base
math.log10()	Returns the base-10 logarithm of x
math.log1p()	Returns the natural logarithm of 1+x
math.log2()	Returns the base-2 logarithm of x
math.perm()	Returns the number of ways to choose k items from n items with order and without repetition
math.pow()	Returns the value of x to the power of y

math.prod()	Returns the product of all the elements in an iterable
math.radians()	Converts a degree value into radians
math.remainder()	Returns the closest value that can make numerator completely divisible by the denominator
math.sin()	Returns the sine of a number
math.sinh()	Returns the hyperbolic sine of a number
math.sqrt()	Returns the square root of a number
math.tan()	Returns the tangent of a number
math.tanh()	Returns the hyperbolic tangent of a number
math.trunc()	Returns the truncated integer parts of a number

Math Constants

Constant	Description
math.e	Returns Euler's number (2.7182...)
math.inf	Returns a floating-point positive infinity
math.nan	Returns a floating-point NaN (Not a Number) value
math.pi	Returns PI (3.1415...)
math.tau	Returns tau (6.2831...)

Python math.pow() Method

Find the value of 9 raised to the power of 3:

Example

```
# Import math Library
import math
#Return the value of 9 raised to the power of 3
print(math.pow(9, 3))
```

The **math.pow() method** returns the value of x raised to power y.
If x is negative and y is not an integer, it returns a ValueError.
This method converts both arguments into a float.
Tip: If we use math.pow(1.0,x) or math.pow(x,0.0), it will always returns 1.0.

Syntax
math.pow(*x*, *y*)
Parameter Values

Parameter	Description
x	Required. A number which represents the base
y	Required. A number which represents the exponent

Technical Details

Return Value:	A float value, representing the value of x to the power of y (x^y)

Python Random randint() Method

Example
Return a number between 3 and 9 (both included):

```
import random

print(random.randint(3, 9))
```

The randint() method returns an integer number selected element from the specified range.

Note: This method is an alias for randrange(start, stop+1).

Syntax
random.randint(*start*, *stop*)
Parameter Values

Parameter	Description
start	Required. An integer specifying at which position to start.
stop	Required. An integer specifying at which position to end.

Python math.sqrt() Method
Example

Find the square root of different numbers:

```
# Import math Library
import math

# Return the square root of different numbers
print (math.sqrt(9))
```

```
print (math.sqrt(25))
print (math.sqrt(16))
```

Output

```
3.0
5.0
4.0
```

Python math.trunc() Method

Find the integer part of different numbers:
Example

```
# Import math Library
import math

# Return the truncated integer parts of different numbers
print(math.trunc(2.77))
print(math.trunc(8.32))
print(math.trunc(-99.29))
```

Output

```
2
8
-99
```

Lecture No. 12

Python Functions

A function is a block of code which only runs when it is called.

We can pass data, known as parameters, into a function.

A function can return data as a result.

Creating a Function

In Python a function is defined using the def keyword:

Example

```
def my_function():
  print("Hello from a function")
```

Calling a Function

To call a function, use the function name followed by parenthesis:

Example

```
def my_function():
  print("Hello from a function")

my_function()
```

Arguments

Information can be passed into functions as arguments.

Arguments are specified after the function name, inside the parentheses. You can add as many arguments as you want, just separate them with a comma.

The following example has a function with one argument (fname). When the function is called, we pass along a first name, which is used inside the function to print the full name:

Example

```
def my_function(fname):
  print(fname + " Refsnes")

my_function("Emil")
my_function("Tobias")
my_function("Linus")
```

Parameters or Arguments?
The terms parameter and argument can be used for the same thing: information that are passed into a function.

From a function's perspective:

- A parameter is the variable listed inside the parentheses in the function definition.
- An argument is the value that is sent to the function when it is called.

Number of Arguments
By default, a function must be called with the correct number of arguments. Meaning that if your function expects 2 arguments, you have to call the function with 2 arguments, not more, and not less.

Example
This function expects 2 arguments, and gets 2 arguments:

```
def my_function(fname, lname):
  print(fname + " " + lname)

my_function("Emil", "Refsnes")
```

If you try to call the function with 1 or 3 arguments, you will get an error:
ExampleGet your own Python Server
This function expects 2 arguments, but gets only 1:

```
def my_function(fname, lname):
  print(fname + " " + lname)

my_function("Emil")
```

Arbitrary Arguments, *args

If you do not know how many arguments that will be passed into your function, add a * before the parameter name in the function definition.

This way the function will receive a tuple of arguments, and can access the items accordingly:

ExampleGet your own Python Server
If the number of arguments is unknown, add a * before the parameter name:

```
def my_function(*kids):
  print("The youngest child is " + kids[2])

my_function("Emil", "Tobias", "Linus")
```

Arbitrary Arguments are often shortened to *args in Python documentations.

Keyword Arguments
You can also send arguments with the key = value syntax.

This way the order of the arguments does not matter.

Example

```
def my_function(child3, child2, child1):
  print("The youngest child is " + child3)

my_function(child1 = "Emil", child2 = "Tobias", child3 = "Linus")
```

The phrase Keyword Arguments are often shortened to kwargs in Python documentations.

This way the function will receive a dictionary of arguments, and can access the items accordingly:

Example

If the number of keyword arguments is unknown, add a double ** before the parameter name:

```
def my_function(**kid):
  print("His last name is " + kid["lname"])

my_function(fname = "Tobias", lname = "Refsnes")
```

Default Parameter Value

The following example shows how to use a default parameter value.

If we call the function without argument, it uses the default value:

Example

```
def my_function(country = "Norway"):
  print("I am from " + country)

my_function("Sweden")
my_function("India")
my_function()
my_function("Brazil")
```

Passing a List as an Argument

You can send any data types of argument to a function (string, number, list, dictionary etc.), and it will be treated as the same data type inside the function.

E.g. if you send a List as an argument, it will still be a List when it reaches the function:

Example

```
def my_function(food):
  for x in food:
    print(x)

fruits = ["apple", "banana", "cherry"]

my_function(fruits)
```

Return Values
To let a function return a value, use the return statement:

ExampleGet your own Python Server
```
def my_function(x):
  return 5 * x

print(my_function(3))
print(my_function(5))
print(my_function(9))
```

The pass Statement
function definitions cannot be empty, but if you for some reason have a function definition with no content, put in the pass statement to avoid getting an error.

Example
```
def myfunction():
  pass
```

Recursion
Python also accepts function recursion, which means a defined function can call itself.

Recursion is a common mathematical and programming concept. It means that a function calls itself. This has the benefit of meaning that you can loop through data to reach a result.

In this example, tri_recursion() is a function that we have defined to call itself ("recurse"). We use the k variable as the data, which decrements (-1) every time we recurse. The recursion ends when the condition is not greater than 0 (i.e. when it is 0).

Example
Recursion Example

```
def tri_recursion(k):
  if(k > 0):
    result = k + tri_recursion(k - 1)
    print(result)
  else:
    result = 0
  return result

print("\n\nRecursion Example Results")
tri_recursion(6)
```

Python Lambda

A lambda function is a small anonymous function.
A lambda function can take any number of arguments, but can only have one expression.

Syntax
lambda *arguments* : *expression*
The expression is executed and the result is returned:

Example
Add 10 to argument a, and return the result:

```
x = lambda a : a + 10
print(x(5))
```

Lambda functions can take any number of arguments:
Example
Multiply argument a with argument b and return the result:

```
x = lambda a, b : a * b
print(x(5, 6))
```

Example
Summarize argument a, b, and c and return the result:

```
x = lambda a, b, c : a + b + c
print(x(5, 6, 2))
```

Why Use Lambda Functions?
The power of lambda is better shown when you use them as an anonymous function inside another function.
Say you have a function definition that takes one argument, and that argument will be multiplied with an unknown number:

```
def myfunc(n):
  return lambda a : a * n
```

Use that function definition to make a function that always doubles the number you send in:

Example

```
def myfunc(n):
  return lambda a : a * n

mydoubler = myfunc(2)

print(mydoubler(11))
```

Or, use the same function definition to make a function that always *triples* the number you send in:

Example

```
def myfunc(n):
  return lambda a : a * n

mytripler = myfunc(3)

print(mytripler(11))
```

Or, use the same function definition to make both functions, in the same program:

ExampleGet your own Python Server

```
def myfunc(n):
  return lambda a : a * n

mydoubler = myfunc(2)
mytripler = myfunc(3)

print(mydoubler(11))
print(mytripler(11))
```

Use lambda functions when an anonymous function is required for a short period of time.

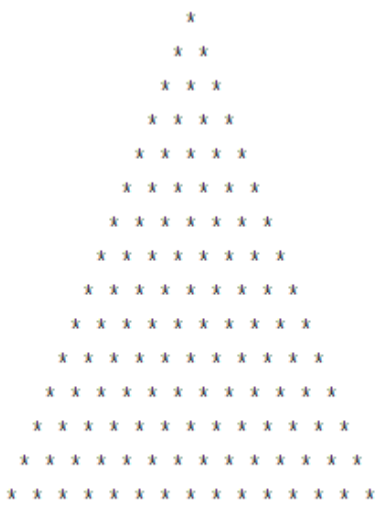
Lecture No. 13

```
def pattern(n):
    k = 2 * n - 2
    for i in range(0,n):
        for j in range(0,k):
            print(end=" ")
        k = k - 1
        for j in range(0, i+1):
            print("*", end=" ")
        print("r")

pattern(15)
```

Output:python pyramid - python for loop- edureka

In the above example, we were able to ma



```
Process finished with exit code 0
```

#for loop example

```
sequence = [3,2,1,4,5]
for var in sequence:
    print(i)

# output
# 3
# 2
```

```
# 1
# 4
# 5
```

#it iterated over the each elements of sequence

Python Program to Print Multiples of 2

```
for i in range(0,11):
    result = i*2
    print(result)
```

Output
0
2
4
6
7
8
9
10

#Nested for loop example

```
for x in range(0,3):

        for y in range(0, 3):

                print(x, "-", y)
```

Output
0 - 0
0 - 1
0 - 2
1 - 0
1 - 1
1 - 2
2 - 0
2 - 1
2 – 2

Example:

```
name = "Adams"
entries = ["Sadm", "Downy", "Downs",]
for x in entries:
    if x == name:
        print("Name Found")
```

```
        break
else:
   print("Nothing found")

# output
# Nothing found


#While loop example
x = 0
while(x < 11):
    print(x*2)
    x += 1
```

Output

```
 0
 2
 4
6
8
10
```

#nested while loop example

```
x = 0
y = 0
while(x > 3):
   while(y>3):
      print(x +"-"+y)
```

Output

```
0 - 0
0 - 1
0 - 2
1 - 0
1 - 1
1 - 2
2 - 0
2 - 1
2 - 2
```

Note: If the condition inside while remains always true, then in that case an infinite loop will execute that will never stop execution.

Write a Python program to construct the following pattern, using a nested for loop.

```
*
* *
* * *
* * * *
* * * * *
* * * *
* * *
* *
*
```

Python Exercise: Construct a specified pattern, using a nested for loop

```
n=5;
for i in range(n):
    for j in range(i):
        print ('* ', end="")
    print('')

for i in range(n,0,-1):
    for j in range(i):
        print('* ', end="")
    print('')
```

Sample Output:

```
*
* *
* * *
* * * *
* * * * *
* * * *
* * *
* *
*
```

Write a Python program to create the multiplication table (from 1 to 10) of a number.

```
n = int(input("Input a number: "))

# use for loop to iterate 10 times
```

```
for i in range(1,11):
   print(n,'x',i,'=',n*i)
```

Sample Output:

```
Input a number: 5
5 x 1 = 5
5 x 2 = 10
5 x 3 = 15
5 x 4 = 20
5 x 5 = 25
5 x 6 = 30
5 x 7 = 35
5 x 8 = 40
5 x 9 = 45
5 x 10 = 50
```

Write a Python program to print the alphabet pattern 'U'.

```
result_str="";
for row in range(0,7):
    for column in range(0,7):
        if (((column == 1 or column == 5) and row != 6) or (row == 6 and column > 1 and column < 5)):
            result_str=result_str+"*"
        else:
            result_str=result_str+" "
    result_str=result_str+"\n"
print(result_str);
```

Sample Output:

```
*   *
*   *
*   *
*   *
*   *
*   *
 ***
```

Lecture No. 14

Python User-Defined Functions

What Are User-Defined Functions In Python?

Functions that we define ourselves to do the certain specific task are referred to as user-defined functions. The way in which we define and call functions in Python are already discussed.

Functions that readily come with Python are called built-in functions. If we use functions written by others in the form of the library, it can be termed as library functions.

All the other functions that we write on our own fall under user-defined functions. So, our user-defined function could be a library function to someone else.

Advantages of user-defined functions

1. User-defined functions help to decompose a large program into small segments which makes the program easy to understand, maintain and debug.
2. If repeated code occurs in a program. The function can be used to include those codes and execute when needed by calling that function.
3. Programmers working on a large project can divide the workload by making different functions.

Syntax

```
def function_name(argument1, argument2, ...) :
    statement_1
    statement_2
    ....
```

Example

```
# Program to illustrate
# the use of user-defined functions

def add_numbers(x,y):
  sum = x + y
  return sum

num1 = 5
num2 = 6

print("The sum is", add_numbers(num1, num2))
```

Output

Enter a number: 2.4
Enter another number: 6.5
The sum is 8.9

Next up on this Python Functions blog, let us check out how we can create a simple application using Python.

Create a function with variable length of arguments

Write a program to create function func1() to accept a variable length of arguments and print their value.

Note: Create a function in such a way that we can pass any number of arguments to this function, and the function should process them and display each argument's value.

Function call:

```
# call function with 3 arguments
func1(20, 40, 60)

# call function with 2 arguments
func1(80, 100)
```

Example

```
def func1(*args):
    for i in args:
        print(i)

func1(20, 40, 60)
func1(80, 100)
```

Output

```
20
40
60
80
100
```

Write a program to create a recursive function to calculate the sum of numbers from 0 to 10.

A recursive function is a function that calls itself again and again.

```
def addition(num):
    if num:
        # call same function by reducing number by 1
        return num + addition(num - 1)
    else:
        return 0

res = addition(10)
print(res)
```

Output

```
55
```

Find the largest item from a given list

```
x = [4, 6, 8, 24, 12, 2]
print(max(x))
```

Output
24

Write a Python program to reverse a string.

Sample String: "1234abcd"
Expected Output: "dcba4321"

```
def string_reverse(str1):

    rstr1 = ''
    index = len(str1)
    while index > 0:
        rstr1 += str1[ index - 1 ]
        index = index - 1
    return rstr1
print(string_reverse('1234abcd'))
```

Sample Output:

dcba4321

Write a Python function that prints out the first n rows of Pascal's triangle.

Note : Pascal's triangle is an arithmetic and geometric figure first imagined by Blaise Pascal.

Each number is the two numbers above it added together

```
def pascal_triangle(n):
   trow = [1]
   y = [0]
   for x in range(max(n,0)):
      print(trow)
      trow=[l+r for l,r in zip(trow+y, y+trow)]
   return n>=1
pascal_triangle(6)
```

Sample Output:

[1]
[1, 1]
[1, 2, 1]
[1, 3, 3, 1]
[1, 4, 6, 4, 1]
[1, 5, 10, 10, 5, 1]

Write a Python function to check whether a number is "Perfect" or not.

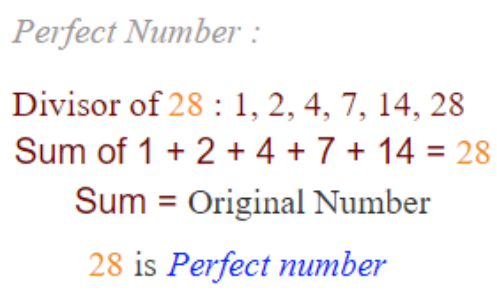
A perfect number is a positive integer that is equal to the sum of its proper positive divisors, that is, the sum of its positive divisors excluding the number itself (also known as its aliquot sum). Equivalently, a perfect number is a number that is half the sum of all of its positive divisors (including itself).

Example : The first perfect number is 6, because 1, 2, and 3 are its proper positive divisors, and 1 + 2 + 3 = 6. Equivalently, the number 6 is equal to half the sum of all its positive divisors: (1 + 2 + 3 + 6) / 2 = 6. The next perfect number is 28 = 1 + 2 + 4 + 7 + 14. This is followed by the perfect numbers 496 and 8128.

```
def perfect_number(n):
    sum = 0
    for x in range(1, n):
        if n % x == 0:
            sum += x
    return sum == n
print(perfect_number(6))
```

Sample Output:

True



Lecture No. 15

What is Regular Expression?

In the real world, string parsing in most programming languages is handled by regular expression. Regular expression in a python programming language is a method used for matching text pattern.
The "re" module which comes with every python installation provides regular expression support.
In python, a regular expression search is typically written as:

match = re.search(pattern, string)

The re.search() method takes two arguments, a regular expression pattern and a string and searches for that pattern within the string. If the pattern is found within the string, search() returns a match object or None otherwise. So in a regular expression, given a string, determine whether that string matches a given pattern, and, optionally, collect substrings that contain relevant information.

A regular expression can be used to answer questions like –

- Is this string a valid URL?
- Which users in /etc/passwd are in a given group?
- What is the date and time of all warning messages in a log file?
- What username and document were requested by the URL a visitor typed?

Matching patterns

Regular expressions are complicated mini-language. They realy on special characters to match unknown strings, but let's start with literal characters, such as letters, numbers, and the space character, which always match themselves. **Let's see a basic example:**

```
#Need module 're' for regular expression
import re
#
search_string = "TutorialsPoint"
pattern = "Tutorials"
match = re.match(pattern, search_string)
#If-statement after search() tests if it succeeded
if match:
   print("regex matches: ", match.group())
else:
   print('pattern not found')
```

Result

regex matches: Tutorials

Matching a string

The "re" module of python has numerous method, and to test whether a particular regular expression matches a specific string, you can use re.search(). The re.MatchObject provides additional information like which part of the string the match was found.

Syntax

matchObject = re.search(pattern, input_string, flags=0)

Example

```
#Need module 're' for regular expression

import re
# Lets use a regular expression to match a date string.
regex = r"([a-zA-Z]+) (\d+)"
if re.search(regex, "Jan 2"):
  match = re.search(regex, "Jan 2")
  # This will print [0, 5), since it matches at the beginning and end of the
  # string
  print("Match at index %s, %s" % (match.start(), match.end()))
  # The groups contain the matched values. In particular:
  # match.group(0) always returns the fully matched string
  # match.group(1), match.group(2), ... will return the capture
  # groups in order from left to right in the input string
  # match.group() is equivalent to match.group(0)
  # So this will print "Jan 2"
  print("Full match: %s" % (match.group(0)))
  # So this will print "Jan"
  print("Month: %s" % (match.group(1)))
  # So this will print "2"
  print("Day: %s" % (match.group(2)))
else:
  # If re.search() does not match, then None is returned
  print("Pattern not Found! ")
```

Result

```
Match at index 0, 5
Full match: Jan 2
Month: Jan
Day: 2
```

As the above method stops after the first match, so is better suited for testing a regular expression than extracting data.

Capturing Groups

If the pattern includes two or more parenthesis, then the end result will be a tuple instead of a list of string, with the help of parenthesis() group mechanism and finall(). Each pattern matched is represented by a tuple and each tuple contains group(1), group(2).. data.

```
for tuple in matches:
  print("Username: ",tuple[0]) #username
  print("Host: ",tuple[1]) #host
```

Result

```
[('john', 'hotmail.com'), ('hello', 'Tutorialspoint.com'), ('python', 'gmail.com')]
Username: john
Host: hotmail.com
Username: hello
Host: Tutorialspoint.com
Username: python
Host: gmail.com
```

Python String replace() Method

Example

Replace the word "bananas":

```
txt = "I like bananas"

x = txt.replace("bananas", "apples")

print(x)
```

Definition and Usage

The replace() method replaces a specified phrase with another specified phrase.

Note: *All* occurrences of the specified phrase will be replaced, if nothing else is specified.

Syntax

string.replace(*oldvalue*, *newvalue*, *count*)

Parameter Values

Parameter	Description
oldvalue	Required. The string to search for
newvalue	Required. The string to replace the old value with
count	Optional. A number specifying how many occurrences of the old value you want to replace. Default is all occurrences

Example

Replace all occurrence of the word "one":

```
txt = "one one was a race horse, two two was one too."

x = txt.replace("one", "three")
```

```
print(x)
```

Example
Replace the two first occurrence of the word "one":

```
txt = "one one was a race horse, two two was one too."

x = txt.replace("one", "three", 2)

print(x)
```

Example

Replace the word "bananas":

```
txt = "I like bananas"

x = txt.replace("bananas", "apples")

print(x)
```

Definition and Usage
The replace() method replaces a specified phrase with another specified phrase.

Note: All occurrences of the specified phrase will be replaced, if nothing else is specified.

Syntax

```
string.replace(oldvalue, newvalue, count)
```

Parameter	**Values**
Parameter	Description
oldvalue	Required. The string to search for
newvalue	Required. The string to replace the old value with
count	Optional. A number specifying how many occurrences of the old value you want to replace. Default is all occurrences

Replace all occurrence of the word "one":

```
txt = "one one was a race horse, two two was one too."

x = txt.replace("one", "three")

print(x)
```

ExampleGet your own Python Server
Replace the two first occurrence of the word "one":

```
txt = "one one was a race horse, two two was one too."
```

```
x = txt.replace("one", "three", 2)

print(x)
```

What is the search() function in Python?

In Python, search() is a method of the module re.

Syntax of search()

re.search(pattern, string):

It is similar to re.match() but it doesn't limit us to find matches at the beginning of the string only. Unlike in re.match() method, here searching for pattern 'Tutorials' in the string 'TP Tutorials Point TP' will return a match.

Example

```
import re
result = re.search(r'Tutorials', 'TP Tutorials Point TP')
print result.group(0)
```

Output

Tutorials

Here you can see that, search() method is able to find a pattern from any position of the string but it only returns the first occurrence of the search pattern.

Lecture No. 16

Function to check prime number

```
def isPrime(n):
    return all([(n % j) for j in range(2, int(n/2)+1)]) and n>1

# Main code
num = 59
if isPrime(num):
  print(num, "is a prime number")
else:
  print(num, "is not a prime number")

num = 7
if isPrime(num):
  print(num, "is a prime number")
else:
  print(num, "is not a prime number")

num = 123
if isPrime(num):
  print(num, "is a prime number")
else:
  print(num, "is not a prime number")
```

Output

```
59 is a prime number
7 is a prime number
123 is not a prime number
```

Program to print all positive numbers in a range

```
# Python program to print all
# positive numbers in a range

# Getting list from user
myList = []
lLimit = int(input("Enter Lower limit of the range : "))
uLimit = int(input("Enter Upper limit of the range : "))

# printing all positive values in a range
print("All positive numbers of the range : ")
for i in range(lLimit, uLimit):
    if i >= 0:
            print(i, end = " ")
```

Output:

```
Enter Lower limit of the range : -5
Enter Upper limit of the range : 10
All positive numbers of the range :
0 1 2 3 4 5 6 7 8 9
```

* Here, the upper value is not included.

Python code to find factorial using recursion

```
def factorial(num):
    # if number is negative - print error
    if num < 0:
        print("Invalid number...")
    # if number is 0 or 1 - the factorial is 1
    elif num == 0 or num == 1:
        return 1
    else:
        # calling function itself i.e. recursive
        return num * factorial(num - 1)

# main code
if __name__ == '__main__':
    #input the number
    x = int(input("Enter an integer number: "))
    print("Factorial of ", x, " is = ", factorial(x))

    x = int(input("Enter another integer number: "))
    print("Factorial of ", x, " is = ", factorial(x))

    x = int(input("Enter another integer number: "))
    print("Factorial of ", x, " is = ", factorial(x))
```

Output

```
Enter an integer number: 5
Factorial of  5  is =  120
Enter another integer number: 0
Factorial of  0  is =  1
Enter another integer number: -3
Invalid number...
Factorial of  -3  is =  None
```

Write a Python function to multiply all the numbers in a list.

```
def multiply(numbers):
    total = 1
    for x in numbers:
        total *= x
    return total
print(multiply((8, 2, 3, -1, 7)))
```

Sample Output:

-336

Python Strings

In computer programming, a string is a sequence of characters. For example, "hello" is a string containing a sequence of characters 'h', 'e', 'l', 'l', and 'o'.

We use single quotes or double quotes to represent a string in Python.

For example,

```
# create a string using double quotes
string1 = "Python programming"
# create a string using single quotes
string1 = 'Python programming'
```

.

Example: Python String

```
# create string type variables

name = "Python"
print(name)

message = "I love Python."
print(message)
```

Output

```
Python
I love Python.
```

In the above example, we have created string-type variables: name and message with values "Python" and "I love Python" respectively.

Access String Characters in Python

We can access the characters in a string in three ways.

Indexing: One way is to treat strings as a list and use index values. For example,

```
greet = 'hello'

# access 1st index element
print(greet[1]) # "e"
```

Negative Indexing: Similar to a list, Python allows negative indexing for its strings. For example,

```
greet = 'hello'

# access 4th last element
```

```
print(greet[-4]) # "e"
```

Run Code

Slicing: Access a range of characters in a string by using the slicing operator colon :. For example,

```
greet = 'Hello'

# access character from 1st index to 3rd index
print(greet[1:4])  # "ell"
```

Note: If we try to access an index out of the range or use numbers other than an integer, we will get errors.

Python Multiline String

We can also create a multiline string in Python. For this, we use triple double quotes """ or triple single quotes '''. For example,

```
# multiline string
message = """
Never gonna give you up
Never gonna let you down
"""

print(message)
```

Output

```
Never gonna give you up
Never gonna let you down
```

In the above example, anything inside the enclosing triple-quotes is one multiline string.

Python String Operations

There are many operations that can be performed with strings which makes it one of the most used data types in Python.

1. Compare Two Strings

We use the == operator to compare two strings. If two strings are equal, the operator returns True. Otherwise, it returns False. For example,

```
str1 = "Hello, world!"
str2 = "I love Python."
str3 = "Hello, world!"

# compare str1 and str2
print(str1 == str2)

# compare str1 and str3
print(str1 == str3)
```

Run Code

Output

False
True

In the above example,

str1 and str2 are not equal. Hence, the result is False.
str1 and str3 are equal. Hence, the result is True.
2. Join Two or More Strings
In Python, we can join (concatenate) two or more strings using the + operator.

```
greet = "Hello, "
name = "Jack"

# using + operator

result = greet + name

print(result)

# Output: Hello, Jack
```

In the above example, we have used the + operator to join two strings: greet and name.

```
greet = 'Hello'
# iterating through greet string
for letter in greet:
    print(letter)
```

Output

```
H
e
l
l
o
```

Python String Length

In Python, we use the len() method to find the length of a string. For example,

```
greet = 'Hello'   # count length of greet string
print(len(greet))

# Output: 5

print('a' in 'program') # True
print('at' not in 'battle') False
```